Taller investigativo: DOM y eventos en JavaScript

Programa: Análisis y Desarrollo de Software (ADSO)

Nombre: Karen Yuliana Gómez Buesaco

Fecha: 09/10/2026

Introducción

En este taller se explica qué es el DOM, cómo JavaScript puede seleccionar y modificar elementos de una página web y cómo se utilizan los eventos para responder a las acciones del usuario. También se presentan ejemplos prácticos que permiten comprobar cómo funcionan estos conceptos en el navegador.

Parte 1. Fundamentos del DOM

1. ¿Qué significa DOM y qué palabras componen esta sigla?

DOM significa Document Object Model, que en español se conoce como Modelo de Objetos del Documento. Es una representación del documento HTML que el navegador organiza para que los lenguajes de programación puedan acceder a sus elementos.

2. ¿Qué es el DOM y para qué sirve en el desarrollo web?

Es una estructura que representa los elementos de una página web como objetos y nodos. Sirve para que JavaScript pueda consultar, agregar, eliminar o modificar contenido, atributos y estilos sin tener que volver a escribir todo el documento HTML.

3. ¿Cómo representa el navegador un documento HTML mediante el DOM?

El navegador organiza el HTML como un árbol. Cada etiqueta se representa como un nodo y los elementos que están dentro de otros aparecen como sus hijos. De esta forma se puede identificar la relación entre el documento, sus elementos y su contenido.

4. ¿Qué relación existe entre HTML, CSS, JavaScript y el DOM?

HTML define la estructura y el contenido; CSS establece la apariencia; JavaScript permite agregar comportamiento e interactividad; y el DOM es la representación de la página que JavaScript puede consultar y modificar.

5. ¿Qué es un nodo en el DOM? Explique la diferencia entre un nodo de elemento, uno de texto y uno de documento.

Un nodo es una parte de la estructura del DOM. Un nodo de elemento corresponde a una etiqueta HTML, por ejemplo <p>. Un nodo de texto contiene texto escrito dentro de un elemento, como “Hola”. El nodo de documento representa el documento completo y sirve como punto de acceso a la estructura de la página.

6. ¿Qué es el árbol DOM? Dibuje un árbol para la estructura HTML indicada.

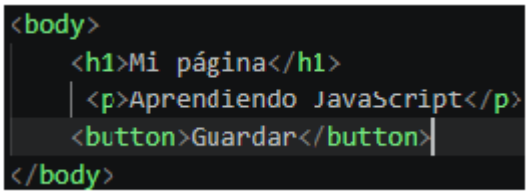
El árbol DOM es la forma jerárquica en la que el navegador organiza los nodos.

7. ¿Qué ocurriría con una página web si JavaScript no pudiera acceder al DOM?

La página todavía podría mostrar su contenido HTML y los estilos CSS, pero sería mucho más difícil cambiar el texto, actualizar partes de la página, responder a clics o agregar elementos de forma dinámica.

8. ¿El DOM es lo mismo que el código HTML escrito por el desarrollador? Justifique con un ejemplo.

No son exactamente lo mismo. El HTML es el código que se escribe y el DOM es la estructura que el navegador construye a partir de ese código. Por ejemplo, si el HTML contiene Hola , JavaScript puede cambiar el texto del nodo del DOM a “Buenos días”. El archivo HTML original no tiene que modificarse para que el cambio se vea en esa ejecución.



Parte 2. Selección y manipulación de elementos

getElementById()

Selecciona un elemento por el valor de su atributo id. Devuelve el elemento encontrado o null si no existe.

Ejemplo:

const titulo = document.getElementById("titulo");
titulo.textContent = "Bienvenidos";

getElementsByClassName()

Busca los elementos que tienen una clase determinada. Devuelve una colección HTML de elementos, que puede contener ninguno, uno o varios.

Ejemplo:

const avisos = document.getElementsByClassName("aviso");
if (avisos.length > 0) avisos[0].textContent = "Aviso actualizado";

getElementsByTagName()

Selecciona elementos por el nombre de su etiqueta HTML. Devuelve una colección HTML.

Ejemplo:

```
const parrafos = document.getElementsByTagName("p");
if (parrafos.length > 0) parrafos[0].style.color = "blue";
```

querySelector()

Busca y devuelve el primer elemento que coincide con un selector CSS. Si no encuentra coincidencias, devuelve null.

Ejemplo:

```
const campo = document.querySelector("#correo");
```

querySelectorAll()

Devuelve una NodeList con todos los elementos que coinciden con un selector CSS. A diferencia de querySelector(), puede devolver varios elementos.

Ejemplo:

```
const botones = document.querySelectorAll(".boton");
botones.forEach(boton => boton.disabled = false);
```

innerHTML

Permite consultar o cambiar el contenido HTML que está dentro de un elemento. Interpreta las etiquetas que se asignan como HTML.

Ejemplo:

```
const caja = document.getElementById("caja");
caja.innerHTML = "<strong>Texto importante</strong>";
```

textContent

Permite leer o cambiar el contenido de texto de un elemento. El texto asignado no se interpreta como etiquetas HTML.

Ejemplo:

```
const mensaje = document.getElementById("mensaje");
mensaje.textContent = "Operación terminada";
```

style

Permite cambiar directamente propiedades CSS de un elemento desde JavaScript. Los nombres CSS compuestos se escriben en camelCase, por ejemplo backgroundColor.

Ejemplo:

```
document.getElementById("panel").style.backgroundColor = "lightblue";
```

classList

Permite trabajar con las clases CSS. add() agrega una clase, remove() la quita, toggle() la agrega o quita según corresponda y contains() comprueba si está presente.

Ejemplo:

```
const panel = document.getElementById("panel");
panel.classList.add("activo");
panel.classList.toggle("oculto");
console.log(panel.classList.contains("activo"));
panel.classList.remove("activo");
```

createElement()

Crea un nuevo elemento HTML en memoria. Para que aparezca en la página, luego debe agregarse a un elemento existente.

Ejemplo:

```
const nuevoParrafo = document.createElement("p");
nuevoParrafo.textContent = "Este párrafo fue creado con JavaScript";
```

appendChild()

Agrega un nodo como hijo al final de otro nodo. Se utiliza, por ejemplo, para insertar un elemento creado con createElement().

Ejemplo:

```
const lista = document.getElementById("lista");
const item = document.createElement("li");
item.textContent = "Nuevo elemento";
lista.appendChild(item);
```

remove()

Elimina del documento el elemento sobre el que se llama.

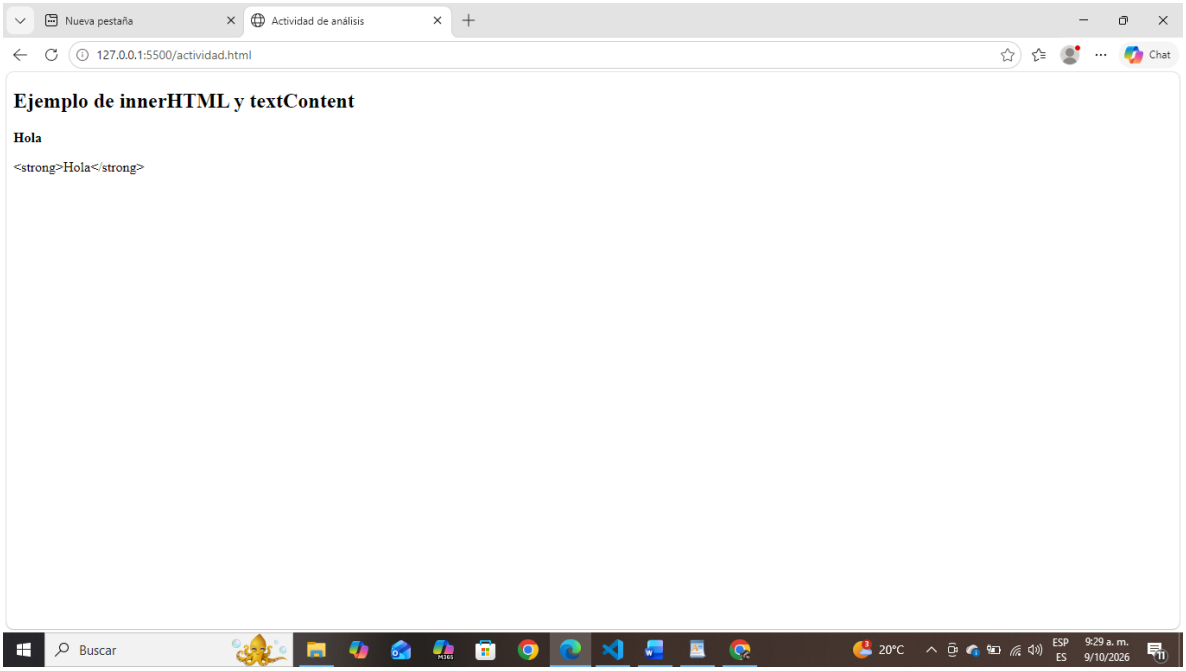
Ejemplo:

```
const aviso = document.getElementById("aviso");
if (aviso) aviso.remove();
```

Actividad de análisis: diferencias entre innerHTML y textContent

La propiedad "innerHTML" permite insertar contenido HTML dentro de un elemento, interpretando las etiquetas que se escriben. En cambio, "textContent" permite insertar texto sin interpretar las etiquetas HTML.

Por ejemplo, si utilizamos "Hola" con "innerHTML", la palabra Hola aparecerá en negrita. Si utilizamos el mismo contenido con "textContent", aparecerán los símbolos de la etiqueta junto con la palabra Hola.



3.1. Conceptos fundamentales

9. ¿Qué es un evento en JavaScript?

Es una acción o suceso que ocurre en la página, como hacer clic, presionar una tecla, escribir en un campo o enviar un formulario.

10. ¿Por qué son importantes los eventos?

Permiten que una página responda a lo que hace el usuario y, por lo tanto, tenga interactividad.

11. ¿Qué relación existe entre un evento, un elemento HTML y una función?

El elemento es donde ocurre la interacción, el evento indica qué ocurrió y la función contiene las instrucciones que se ejecutan como respuesta.

12. ¿Qué significa escuchar o detectar un evento?

Significa registrar una función para que el navegador la ejecute cuando ocurra un tipo de evento en un elemento.

13. ¿Qué es un manejador de eventos?

Es la función que se ejecuta como respuesta a un evento. También se conoce como event handler.

14. ¿Qué diferencia existe entre un evento y una función?

El evento es el suceso, por ejemplo un clic. La función es el conjunto de instrucciones que se ejecutan cuando ese suceso ocurre.

15. ¿Qué es addEventListener() y cuál es su sintaxis general?

Es un método que registra un manejador para un evento en un elemento. Su forma general es elemento.addEventListener("tipoDeEvento", funcion).

16. ¿Qué diferencia hay entre onclick en HTML y addEventListener()?

onclick escrito en HTML mezcla el comportamiento con la estructura y normalmente deja un manejador asignado a esa propiedad. addEventListener() registra el evento desde JavaScript, mantiene el HTML más limpio y permite agregar varios manejadores para el mismo evento.

17. ¿Qué sucede si se registra más de un manejador para el mismo evento y elemento?

Si se agregan con addEventListener(), normalmente se ejecutan todos los manejadores registrados para ese evento, en el orden en que fueron registrados, siempre que no se interrumpa la propagación.

18. ¿Qué es el objeto event y qué información proporciona?

Es un objeto que el navegador entrega al manejador cuando ocurre un evento. Puede incluir el elemento donde se originó, la tecla presionada, las coordenadas del puntero o información sobre el formulario, según el tipo de evento.

3.2. Tipos de eventos

click: Se activa al hacer clic o tocar un elemento. Puede usarse en botones; por ejemplo, al pulsar “Agregar al carrito”.

dblclick: Se activa al hacer doble clic. Puede usarse en una imagen para abrir una vista ampliada.

mouseover: Se activa cuando el puntero entra en un elemento o en uno de sus descendientes. Puede utilizarse para mostrar una ayuda al pasar por una zona.

mouseout: Se activa cuando el puntero sale de un elemento o pasa a uno de sus descendientes. Puede utilizarse para ocultar una ayuda.

mouseenter: Se activa cuando el puntero entra en el elemento; no vuelve a activarse solo por pasar entre sus elementos hijos. Puede destacar una tarjeta.

mouseleave: Se activa cuando el puntero abandona el elemento y no se dispara por moverse entre sus hijos. Puede restaurar el aspecto de una tarjeta.

keydown: Ocurre cuando se presiona una tecla. Puede detectar Enter para enviar una búsqueda.

keyup: Ocurre cuando se suelta una tecla. Puede utilizarse para actualizar una vista después de escribir.

input: Se activa cuando cambia el valor de un campo por una acción del usuario, normalmente mientras escribe. Puede mostrar una vista previa del nombre.

change: Se activa cuando un cambio en un campo queda confirmado; en campos de texto suele ocurrir al cambiar el valor y salir del campo, mientras que en select puede ocurrir al elegir una opción.

submit: Ocurre al intentar enviar un formulario. Puede validar los datos de registro antes de continuar.

focus: Ocurre cuando un elemento recibe el foco, por ejemplo al seleccionar un campo para escribir.

blur: Ocurre cuando un elemento pierde el foco, por ejemplo al salir de un campo.

load: Ocurre cuando un recurso o documento terminó de cargarse. Puede utilizarse para ejecutar una acción cuando una imagen o la página ya esté cargada.

3.3. Preguntas de profundización

19. ¿Por qué un botón puede utilizar click, mientras que un campo de texto puede utilizar input?

Porque el botón suele ejecutar una acción cuando se pulsa, mientras que input permite detectar cada cambio que se hace en el valor del campo mientras la persona escribe.

20. ¿Qué diferencia existe entre mouseenter y mouseover?

mouseenter ocurre cuando el puntero entra en el elemento y no se repite al pasar entre sus elementos hijos. mouseover también puede activarse al pasar de un elemento a uno de sus descendientes.

21. ¿Qué diferencia existe entre keydown y keyup?

keydown ocurre al presionar una tecla y keyup cuando se suelta. Por eso pueden servir para detectar momentos diferentes de la pulsación.

22. ¿Por qué se utiliza preventDefault() con determinados formularios?

Se utiliza para impedir la acción predeterminada del navegador, por ejemplo que el formulario recargue la página al enviarse. Así JavaScript puede revisar los datos y mostrar un mensaje antes de continuar.

23. ¿Qué sucede cuando se ejecuta event.target? Construya un ejemplo.

event.target no es una función: es una propiedad que identifica el elemento donde se originó el evento. Ejemplo: un manejador puede usar console.log(event.target) para mostrar en la consola el botón que se pulsó.

24. ¿Qué diferencia existe entre event.target y event.currentTarget?

target es el elemento donde se originó el evento. currentTarget es el elemento al que está asociado el manejador que se está ejecutando. Pueden ser diferentes cuando el evento se propaga desde un hijo a un padre.

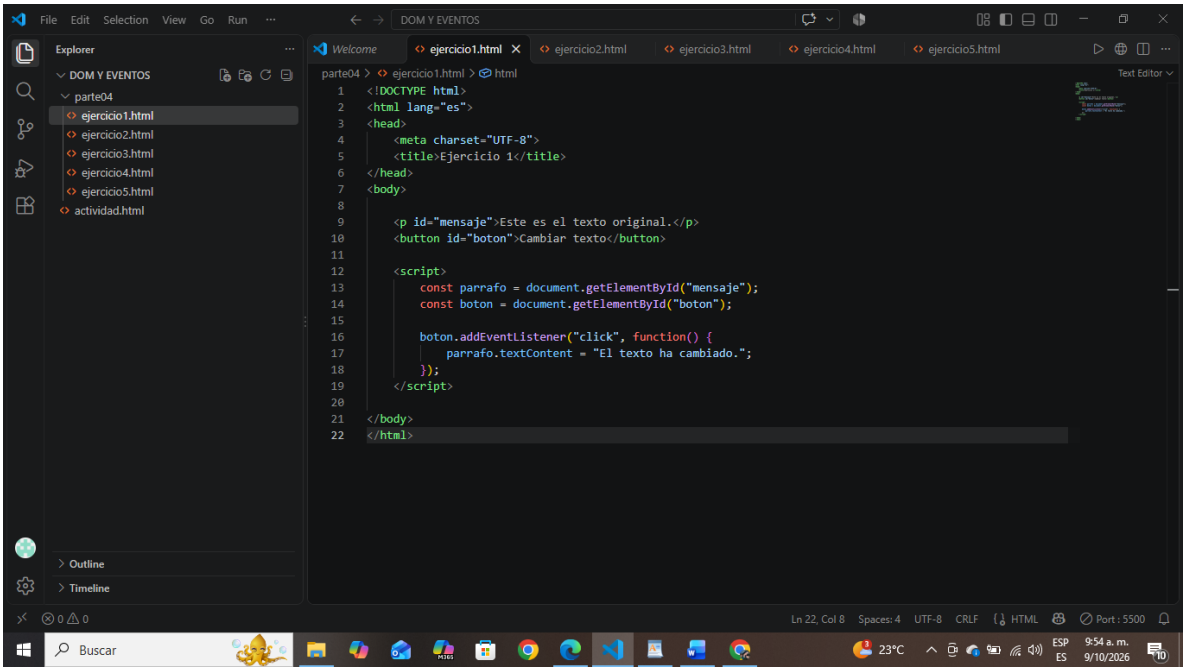
25. ¿Qué son la propagación de eventos, el bubbling y la captura?

La propagación es el recorrido del evento por la estructura de elementos. En bubbling, el evento sube desde el elemento donde ocurrió hacia sus padres. En captura, pasa desde los elementos antecesores hacia el elemento donde se origina.

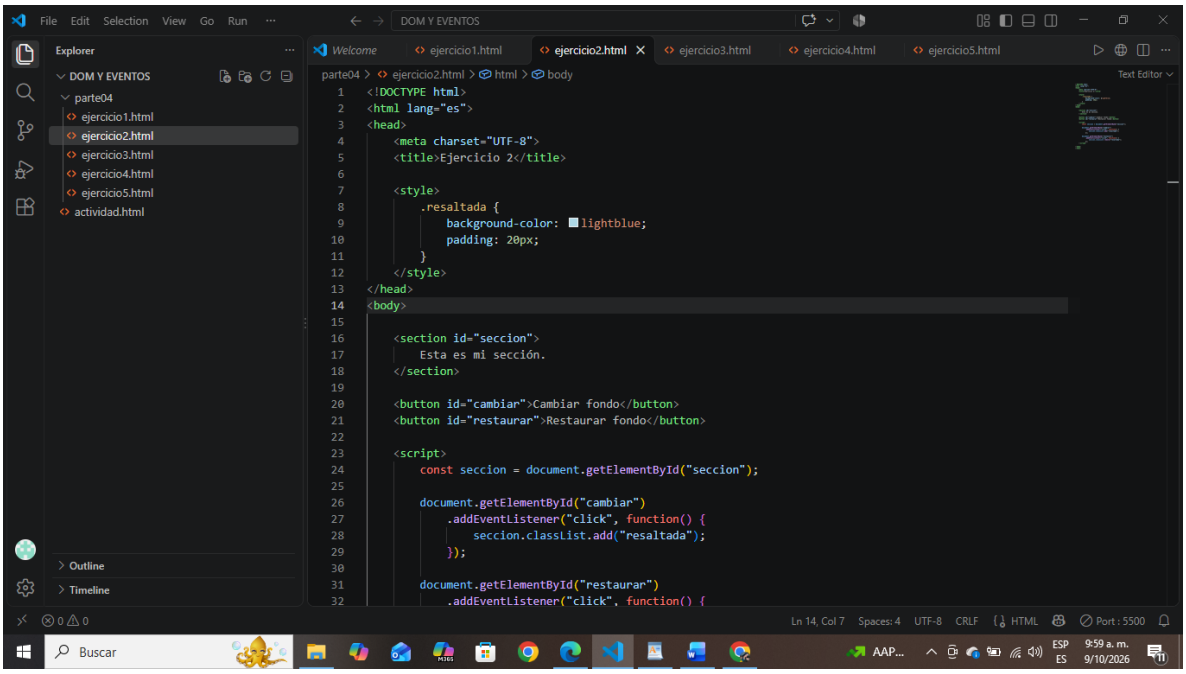
26. ¿En qué consiste la delegación de eventos y cuándo podría ser útil?

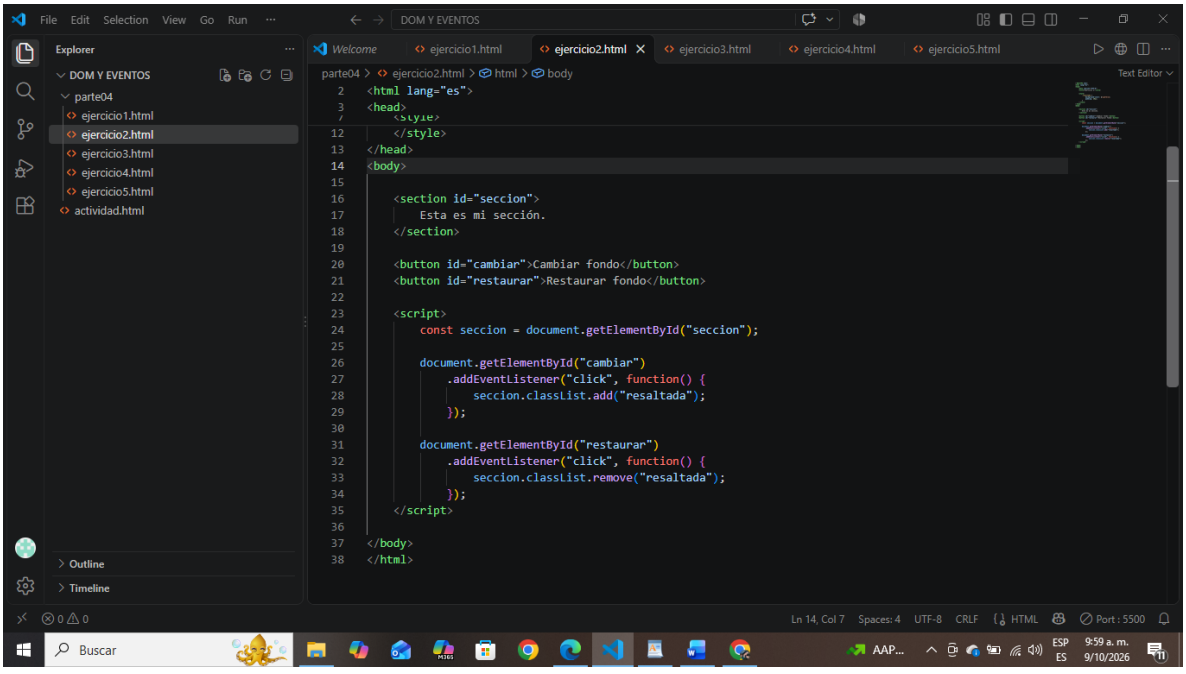
Consiste en poner un solo manejador en un elemento padre para responder a eventos de sus elementos hijos, usando event.target para identificar cuál se activó. Es útil en listas dinámicas, porque también permite atender elementos agregados después.

Parte 4. Taller práctico de investigación

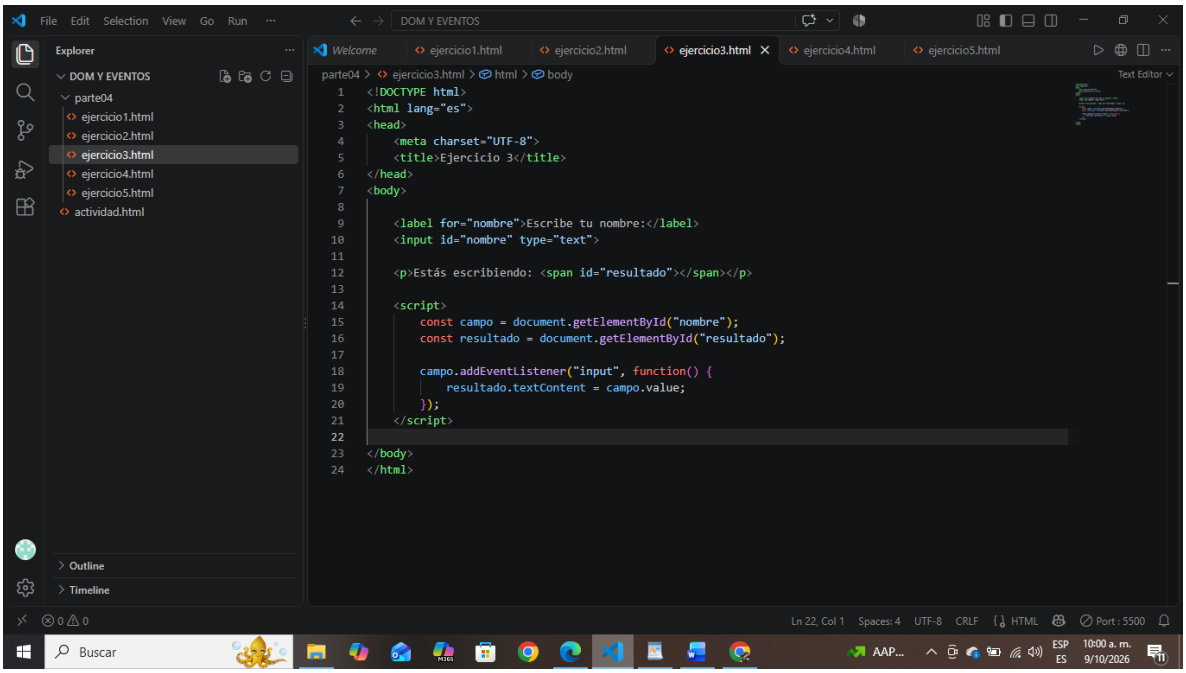


Análisis: seleccioné el párrafo con getElementById("mensaje"). Utilicé el evento click porque el cambio debe ocurrir al presionar el botón. Modifiqué textContent para reemplazar el texto que aparece en el párrafo.

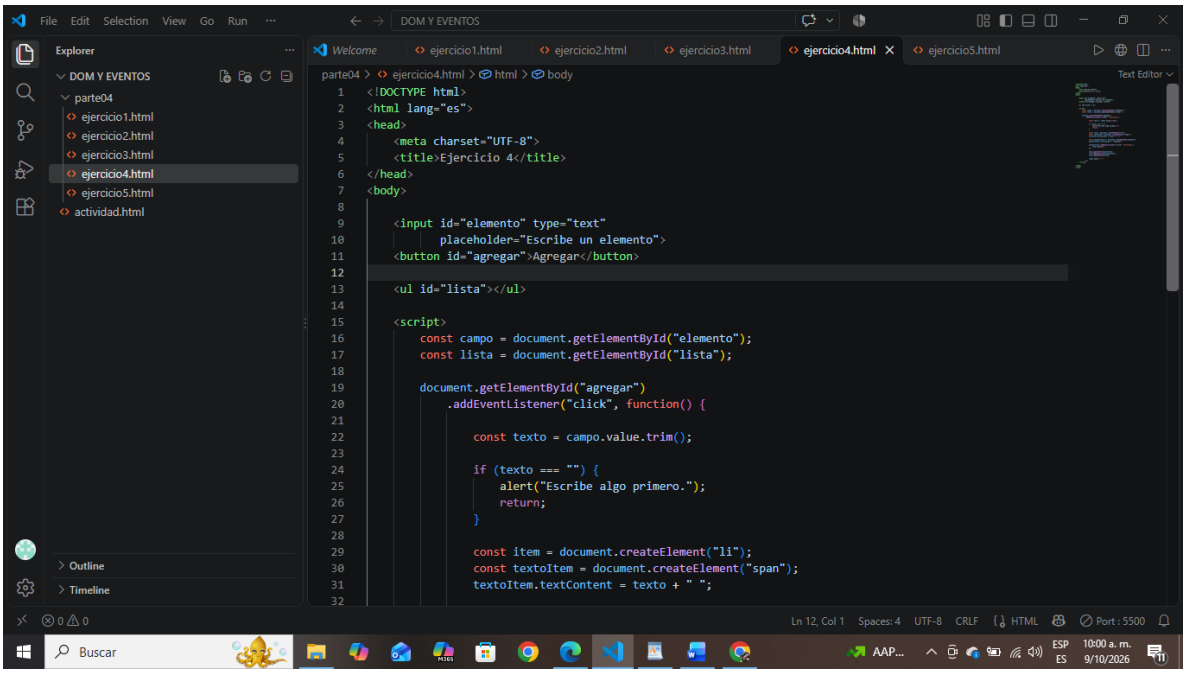


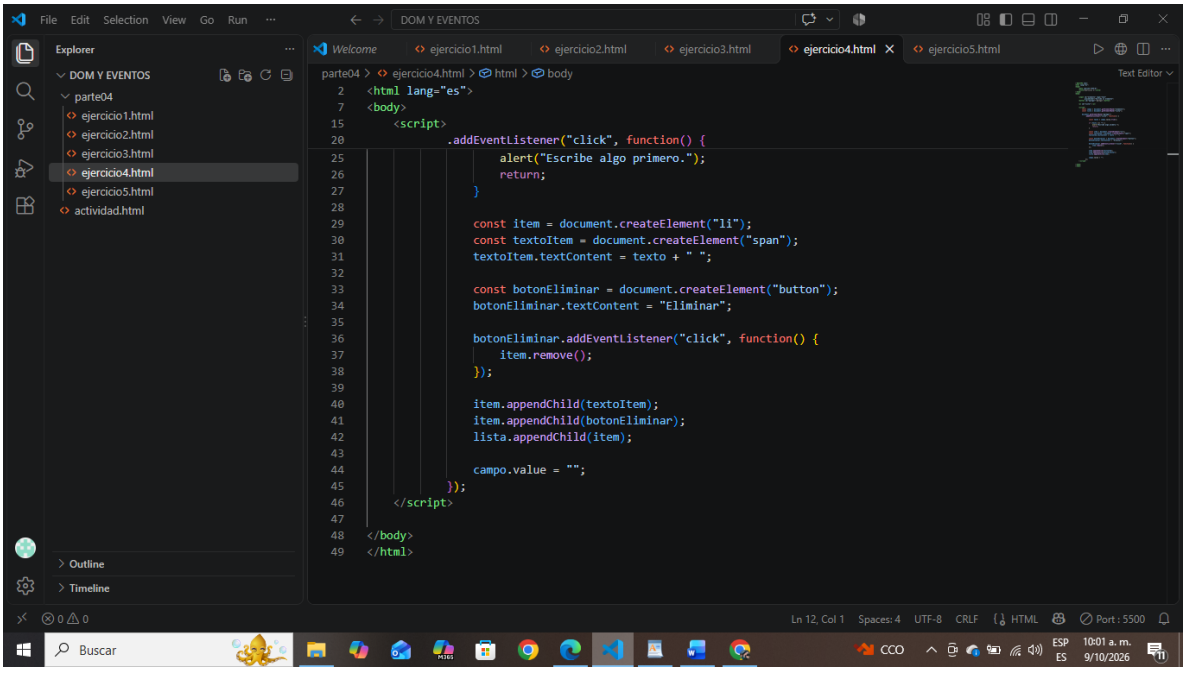


Análisis: para este caso prefiero classList porque los estilos quedan definidos en CSS y JavaScript solo agrega o quita una clase. style también sirve para cambios puntuales, pero classList facilita mantener ordenado el diseño y reutilizar estilos.

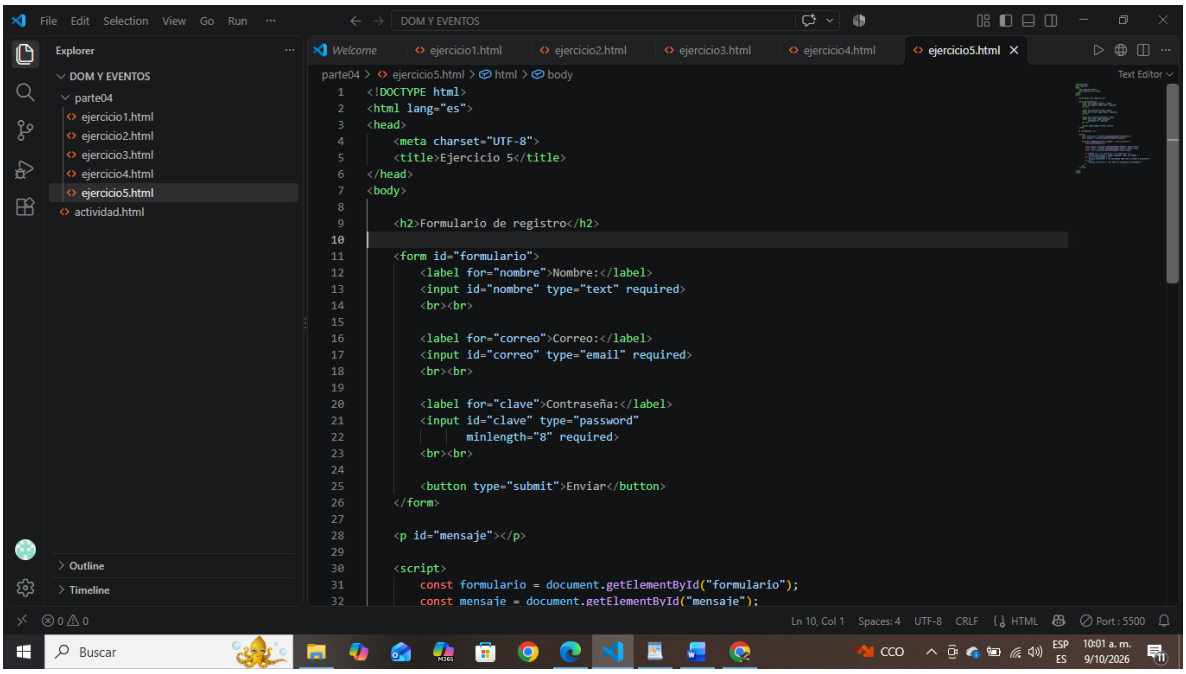


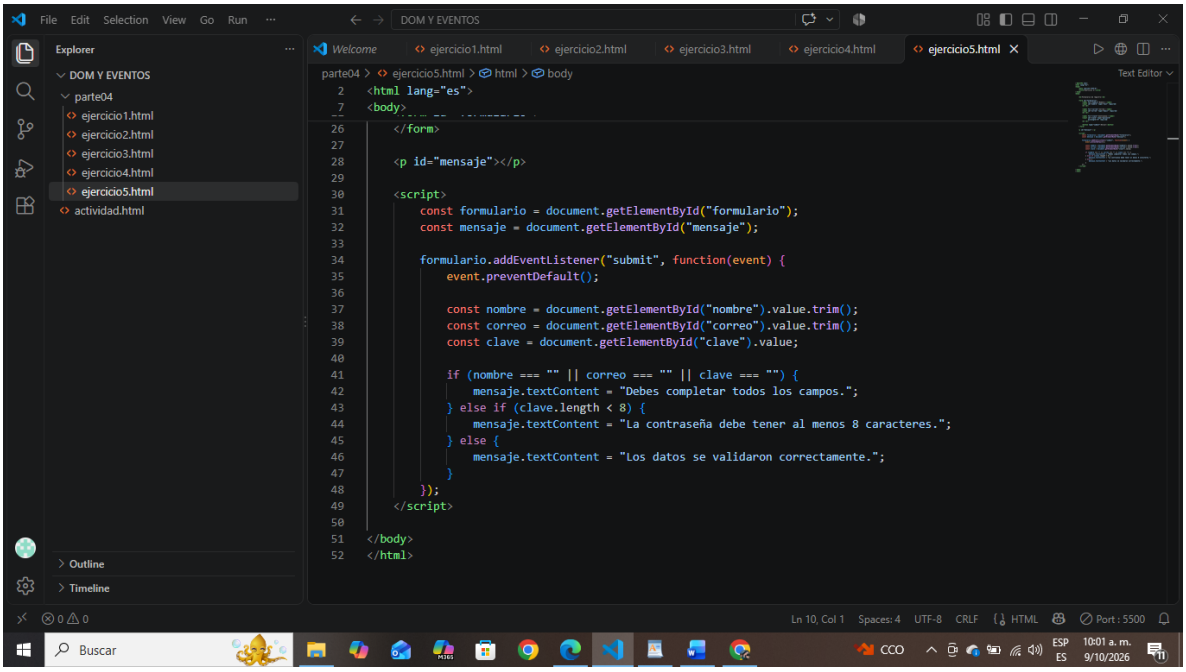
Análisis: input es apropiado porque detecta los cambios mientras se escribe, sin esperar a pulsar un botón. La propiedad value contiene el valor actual del campo de texto.





Análisis: createElement() crea el li, el span y el botón; appendChild() agrega esos nodos a sus elementos correspondientes; remove() elimina el li seleccionado. El evento click se asigna al botón Eliminar cuando se crea, y su función elimina solamente el elemento al que pertenece.





Análisis: submit se activa cuando se intenta enviar el formulario. preventDefault() evita que el navegador lo envíe y recargue la página antes de ejecutar la validación personalizada. Los atributos required, type="email" y minlength realizan comprobaciones básicas desde HTML; JavaScript permite controlar la lógica y los mensajes. En una aplicación real también se debe validar en el servidor.

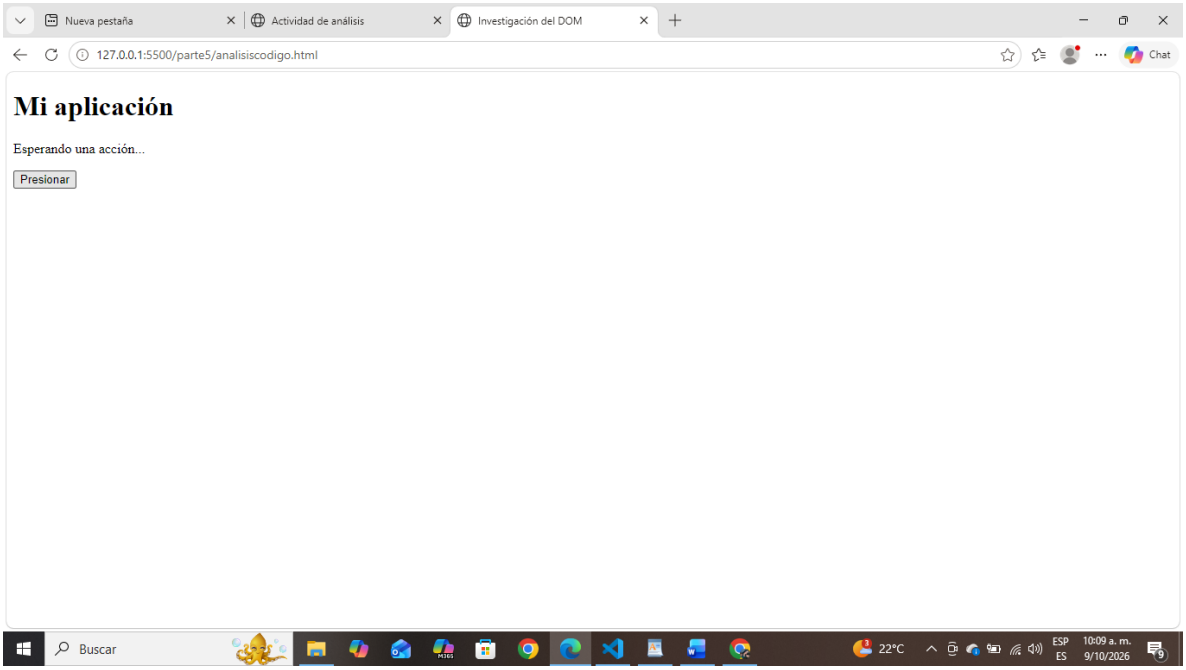
Parte 5. Análisis de código

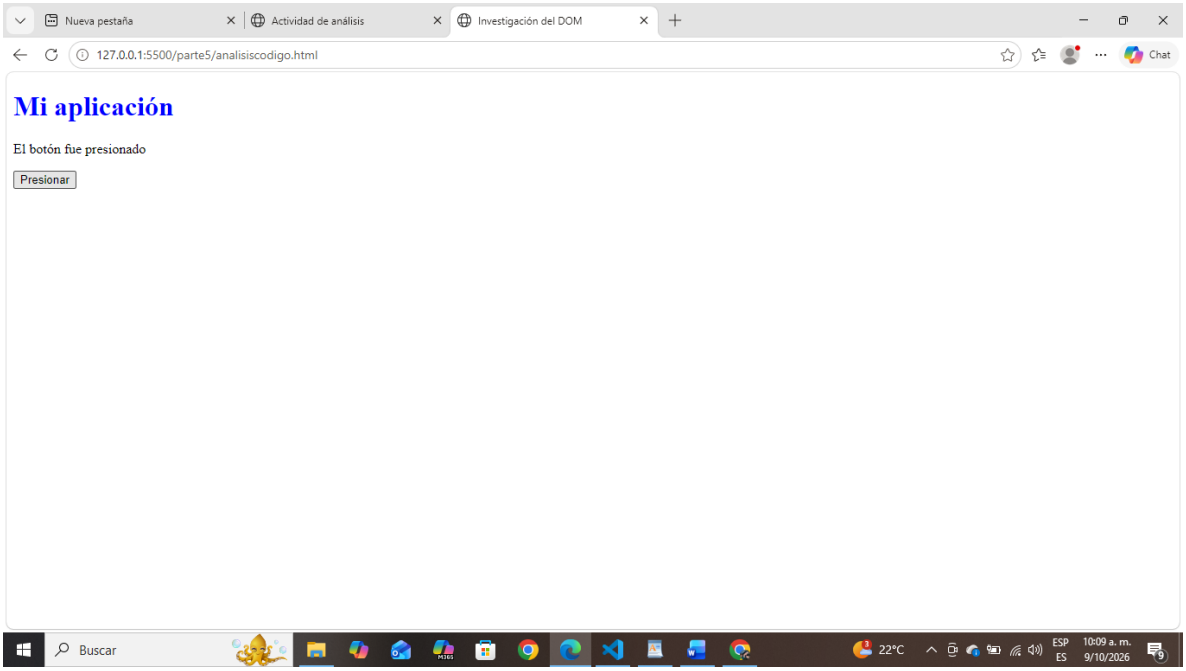
El código de la guía selecciona tres elementos por su id: el título, el párrafo del mensaje y el botón. Cuando se hace clic, cambia el texto del mensaje, pone el título de color azul y muestra en la consola el elemento que originó el evento.

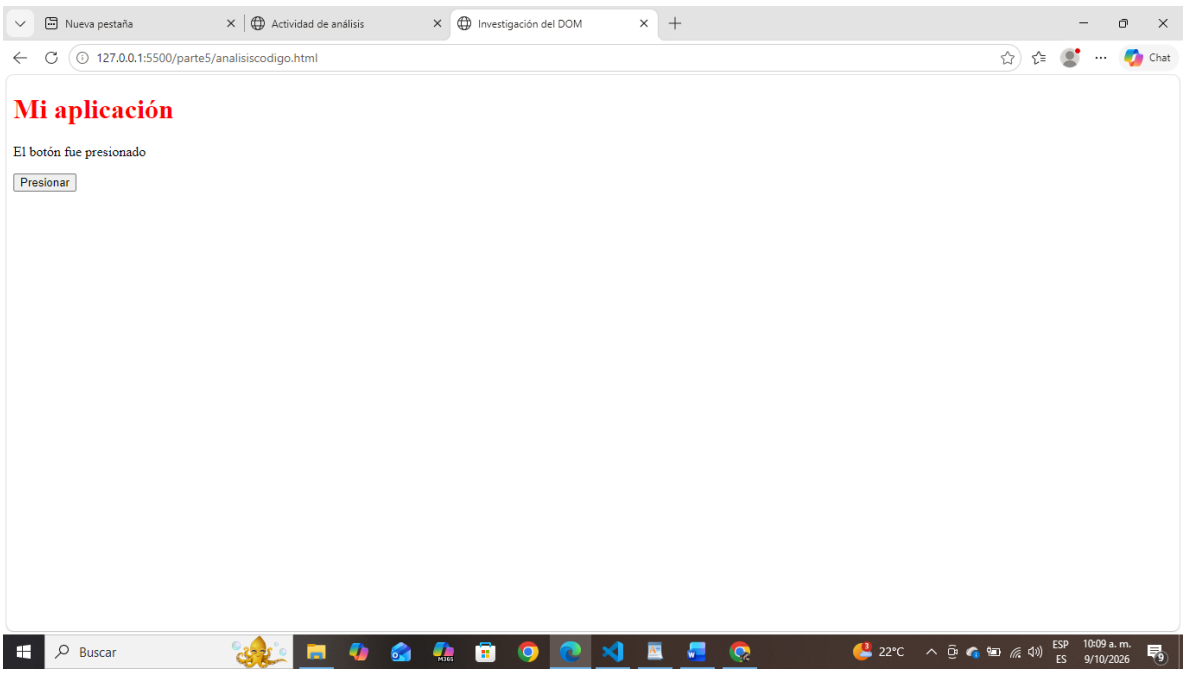
```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Investigación del DOM</title>
</head>
<body>
  <h1 id="titulo">Mi aplicación</h1>
  <p id="mensaje">Esperando una acción...</p>
  <button id="boton">Presionar</button>

  <script>
    const titulo = document.getElementById("titulo");
    const mensaje = document.getElementById("mensaje");
    const boton = document.getElementById("boton");

    boton.addEventListener("click", function(event) {
      mensaje.textContent = "El botón fue presionado";
      titulo.style.color = "blue";
      console.log(event.target);
    });
  </script>
</body>
```







27. ¿Qué elementos se seleccionan mediante getElementById()?

Se seleccionan el título con id="titulo", el párrafo con id="mensaje" y el botón con id="boton".

28. ¿Qué función cumple addEventListener() en este código?

Registra una función para que se ejecute cuando se produzca el evento click en el botón.

29. ¿Qué sucedería si se reemplaza click por dblclick?

La función se ejecutaría cuando se haga doble clic en el botón, en lugar de ejecutarse con un solo clic.

30. ¿Qué cambia cuando se ejecuta la función?

El texto del párrafo pasa a decir “El botón fue presionado” y el color del título cambia a azul. Además, se muestra información del evento en la consola.

31. ¿Qué información muestra console.log(event.target)?

Muestra en la consola el elemento HTML donde se originó el evento. En este ejemplo, normalmente será el botón que se presionó.

32. ¿Qué pasaría si el identificador del botón fuera incorrecto?

getElementById() devolvería null. Al intentar usar addEventListener() sobre ese valor, se produciría un error porque no existe un elemento seleccionado.

33. ¿Qué errores podrían ocurrir si se intenta acceder a un elemento que no existe?

Se puede obtener null y, si después se intenta leer o modificar una propiedad o llamar un método de ese valor, el navegador genera un error de tipo TypeError. También puede ocurrir si el script se ejecuta antes de que el HTML del elemento se haya cargado.

34. Modifique el código para que el título alterne entre azul y rojo.

Código completo modificado:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Investigación del DOM</title>
</head>
<body>
  <h1 id="titulo">Mi aplicación</h1>
  <p id="mensaje">Esperando una acción...</p>
  <button id="boton">Presionar</button>

  <script>
    const titulo = document.getElementById("titulo");
    const mensaje = document.getElementById("mensaje");
    const boton = document.getElementById("boton");
    let estaAzul = false;

    boton.addEventListener("click", function (event) {
      mensaje.textContent = "El botón fue presionado";
      estaAzul = !estaAzul;
      titulo.style.color = estaAzul ? "blue" : "red";
      console.log(event.target);
    });
  </script>
</body>
</html>
```

La variable estaAzul guarda el estado del color. Cada clic invierte su valor; si queda en true, el título se pone azul, y si queda en false, se pone rojo.

35. ¿Cómo comprobaría que el evento funciona sin depender solo de la apariencia visual?

Abriría las herramientas de desarrollador del navegador y revisaría la pestaña Console. El código ejecuta console.log(event.target), así que al presionar el botón debería aparecer el elemento correspondiente. También comprobaría que el mensaje del párrafo cambie.

36. Proponga una mejora para separar JavaScript del HTML.

Crearía un archivo llamado script.js, movería allí el código JavaScript y lo conectaría al HTML con <script src="script.js" defer></script>. El atributo defer hace que el archivo se ejecute después de analizar el HTML, lo que ayuda a que los elementos ya estén disponibles.

Parte 6. Investigación aplicada a un caso real

Sistema seleccionado: Plataforma educativa virtual

1. Nombre y propósito del sistema

Una plataforma educativa virtual es un sistema informático que permite a los estudiantes consultar sus cursos, buscar materiales de estudio y realizar actividades académicas por internet.

2. **Tres elementos de la interfaz con los que interactúa el usuario**

- Botón de consultar cursos: permite mostrar los cursos disponibles.

- Campo de búsqueda: permite escribir el nombre de un curso para encontrarlo más fácilmente.

- Formulario de entrega de actividades: permite enviar las respuestas de una actividad académica.

3. Eventos que se pueden utilizar

- Botón de consultar cursos: evento "click".

- Campo de búsqueda: evento "input".

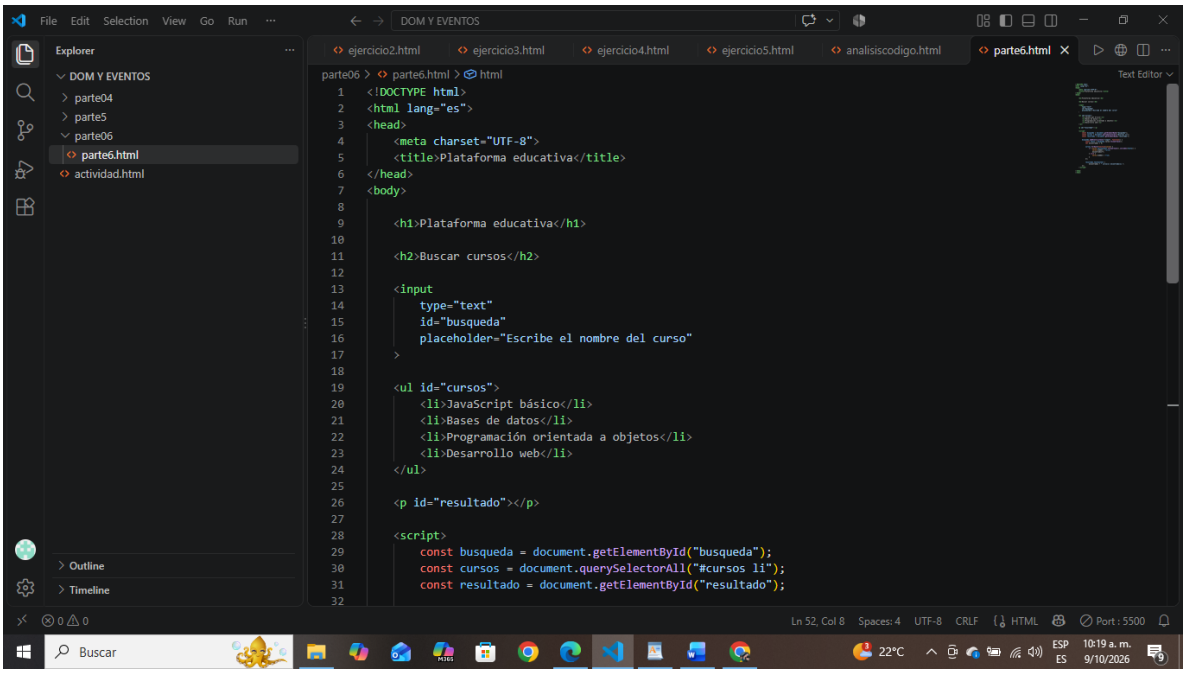
- Formulario de entrega: evento "submit".

4. Acciones que ejecutaría JavaScript

Cuando el estudiante presiona el botón, JavaScript puede mostrar la lista de cursos. Cuando escribe en el campo de búsqueda, puede filtrar los cursos según el texto ingresado. Cuando envía el formulario, puede comprobar que los campos estén completos y mostrar un mensaje de confirmación.

5. Relación con el DOM

El DOM permite que JavaScript acceda a los elementos de la página, como botones, campos de texto, listas y formularios. De esta manera, cuando el estudiante realiza una acción, JavaScript puede modificar el contenido que aparece en pantalla sin necesidad de volver a crear toda la página.



```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Plataforma educativa</title>
</head>
<body>

    <h1>Plataforma educativa</h1>

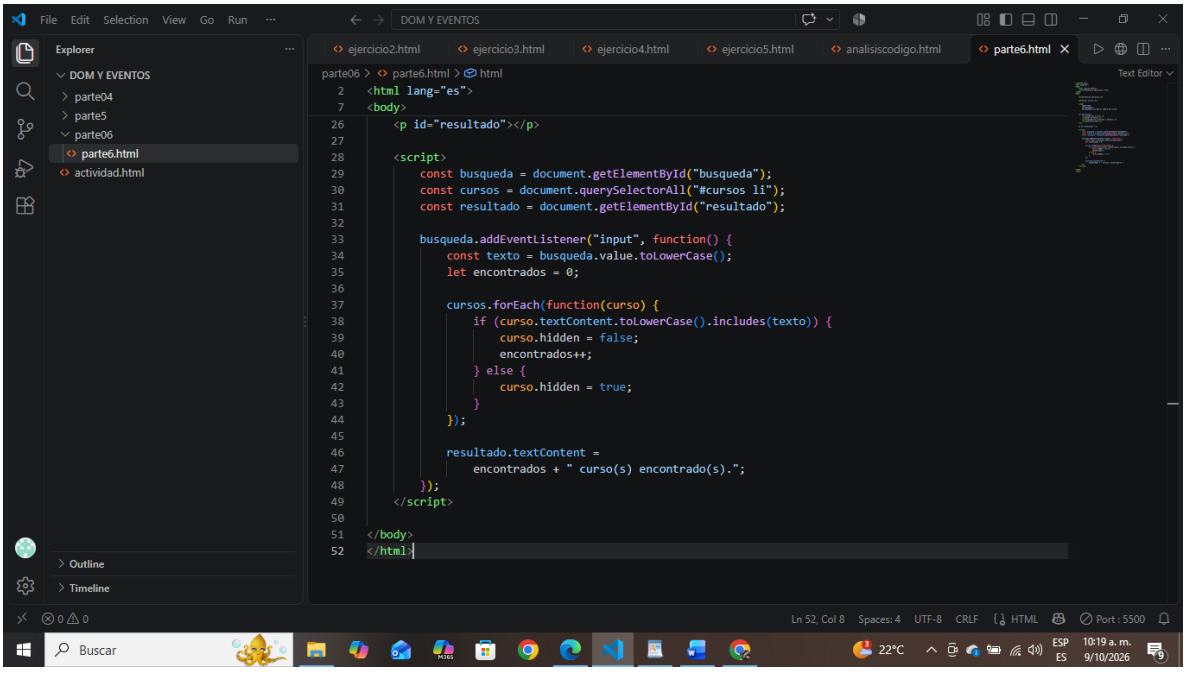
    <h2>Buscar cursos</h2>

    <input
       type="text"
       id="busqueda"
       placeholder="Escribe el nombre del curso"
    >

    <ul id="cursos">
        <li>JavaScript básico</li>
        <li>Bases de datos</li>
        <li>Programación orientada a objetos</li>
        <li>Desarrollo web</li>
    </ul>

    <p id="resultado"></p>

    <script>
        const busqueda = document.getElementById("busqueda");
        const cursos = document.querySelectorAll("#cursos li");
        const resultado = document.getElementById("resultado");
```



```
<html lang="es">
<body>
    <p id="resultado"></p>

    <script>
        const busqueda = document.getElementById("busqueda");
        const cursos = document.querySelectorAll("#cursos li");
        const resultado = document.getElementById("resultado");

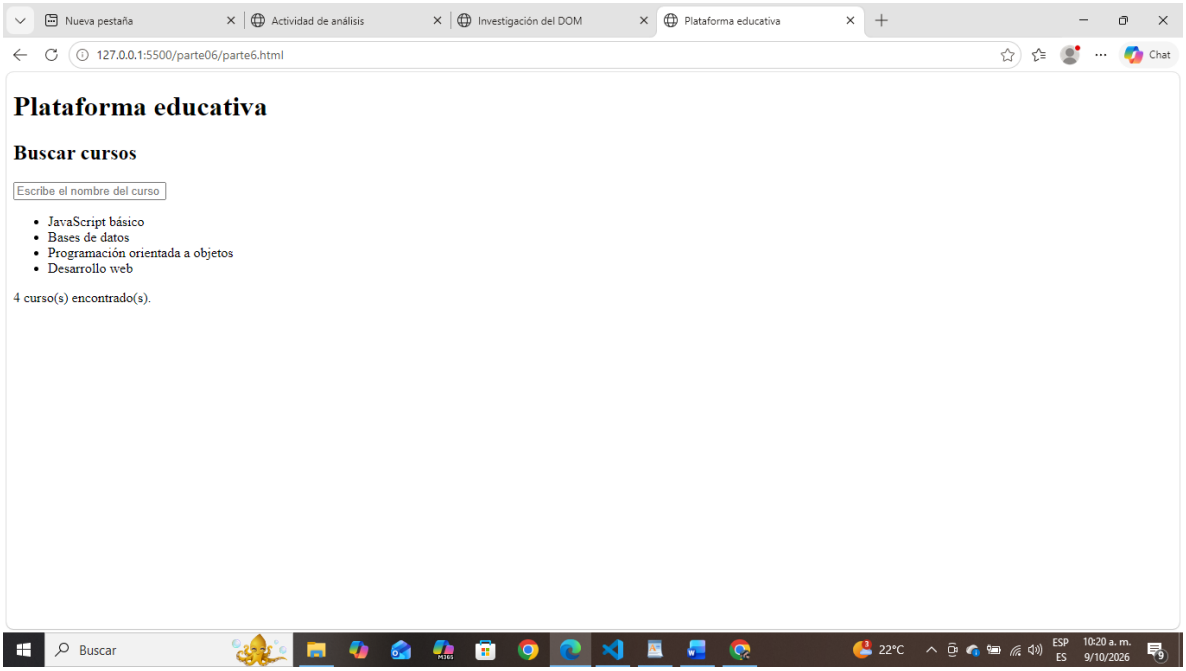
        busqueda.addEventListener("input", function() {
            const texto = busqueda.value.toLowerCase();
            let encontrados = 0;

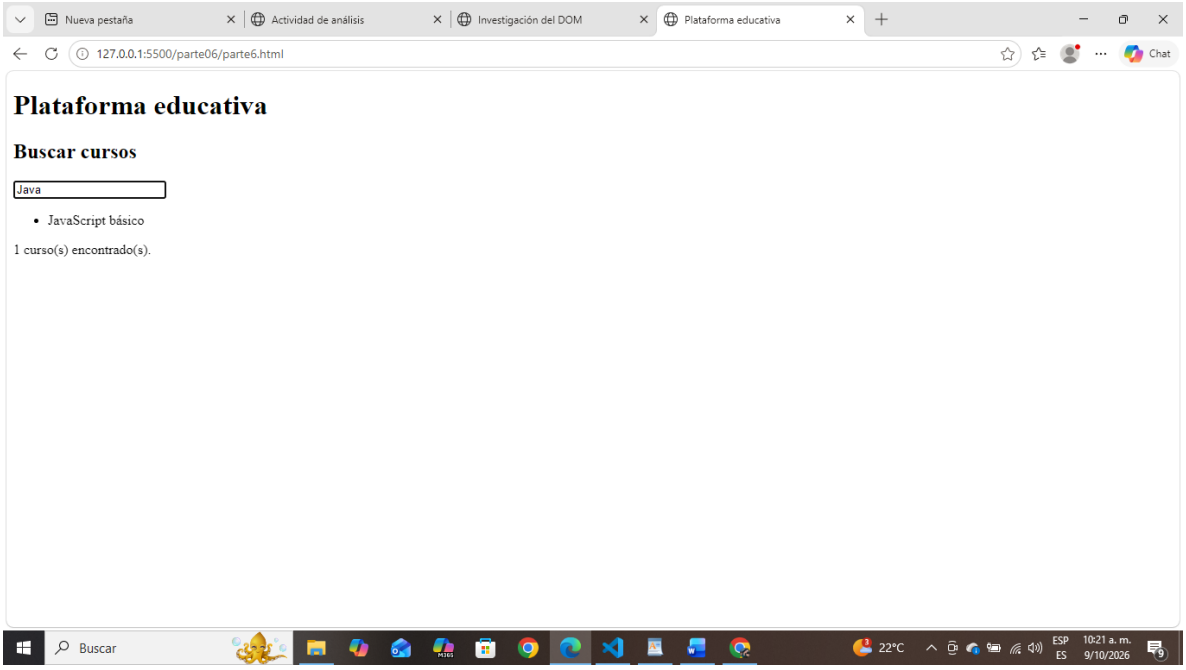
            cursos.forEach(function(curso) {
                if (curso.textContent.toLowerCase().includes(texto)) {
                    curso.hidden = false;
                    encontrados++;
                } else {
                    curso.hidden = true;
                }
            });

            resultado.textContent =
                encontrados + " curso(s) encontrado(s).";
        });
    </script>

</body>
</html>
```

En este prototipo, el evento input detecta lo que escribe el usuario. JavaScript compara el texto con cada curso y utiliza la propiedad hidden para mostrar solamente las coincidencias. El párrafo de estado informa cuántos cursos se encontraron.


Nueva pestaña
Actividad de análisis
Investigación del DOM
Plataforma educativa
127.0.0.1:5500/parte06/parte6.html
Chat
Plataforma educativa
Buscar cursos
Escribe el nombre del curso
JavaScript básico
Bases de datos
Programación orientada a objetos
Desarrollo web
4 curso(s) encontrado(s).
Buscar
22°C
ESP
ES
10:20 a. m.
9/10/2026


Nueva pestaña
Actividad de análisis
Investigación del DOM
Plataforma educativa
127.0.0.1:5500/parte06/parte6.html
Chat
Plataforma educativa
Buscar cursos
Java
JavaScript básico
1 curso(s) encontrado(s).
Buscar
22°C
ESP
ES
10:21 a. m.
9/10/2026